15. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 200

int main() {

    char input[INPUT_SIZE];
    char filename[INPUT_SIZE];

    int file_fd;
    int saved_stdout;
    int saved_stderr;

    printf("Combined Redirection System\n");
    printf("---------------------------\n");

    printf("Enter combined redirection: ");
    fflush(stdout);

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /*
     * We expect:
     *
     * command > filename 2>&1
     *
     */

    char *redirect = strstr(input, ">");

    if (redirect == NULL) {

        printf("Error: '>' operator not found.\n");
        return 1;
    }

    /*
     * Extract filename.
     * Example:
     * command > output.txt 2>&1
     */
    if (sscanf(redirect + 1, "%199s", filename) != 1) {

        printf("Error: Output filename missing.\n");
```

```
  GNU nano 8.7.1
     */
    if (sscanf(redirect + 1, "%199s", filename) != 1) {

        printf("Error: Output filename missing.\n");
        return 1;
    }

    /*
     * Remove possible "2>&1" from filename.
     */
    char *space = strchr(filename, ' ');

    if (space != NULL)
        *space = '\0';

    printf("Output file: %s\n", filename);

    /*
     * Open file for both stdout and stderr.
     */
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

    if (file_fd == -1) {

        perror("Error opening output file");
        return 1;
    }

    printf("File opened successfully.\n");

    /* Save original stdout */
    saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1) {

        perror("dup stdout");
        close(file_fd);
        return 1;
    }

    /* Save original stderr */
    saved_stderr = dup(STDERR_FILENO);

    if (saved_stderr == -1) {

        perror("dup stderr");
        close(file_fd);
        close(saved_stdout);
        return 1;
    }
```

```
    }

    /*
     * Redirect stdout to file.
     */
    if (dup2(file_fd, STDOUT_FILENO) == -1) {

        perror("dup2 stdout");
        return 1;
    }

    /*
     * Redirect stderr to the same file.
     *
     * This is equivalent to:
     *
     * 2>&1
     */
    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1) {

        perror("dup2 stderr");
        return 1;
    }

    close(file_fd);

    /*
     * Generate both normal output
     * and error output.
     */
    printf("Message 1: Standard output.\n");

    fprintf(stderr,
            "Message 2: Standard error.\n");

    printf("Message 3: Standard output.\n");

    fprintf(stderr,
            "Message 4: Standard error.\n");

    printf("Message 5: Standard output.\n");

    fprintf(stderr,
            "Message 6: Standard error.\n");

    /*
     * Flush both streams.
     */
    fflush(stdout);
    fflush(stderr);

    /*
```

```
    /*
     * Restore stdout.
     */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {

        perror("Failed to restore stdout");
        return 1;
    }

    /*
     * Restore stderr.
     */
    if (dup2(saved_stderr, STDERR_FILENO) == -1) {

        perror("Failed to restore stderr");
        return 1;
    }

    close(saved_stdout);
    close(saved_stderr);

    printf("\nStandard streams restored successfully.\n");

    /*
     * Verify integrated output.
     */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {

        perror("Verification failed");
        return 1;
    }

    printf("\nIntegrated Output:\n");
    printf("------------------\n");

    char line[INPUT_SIZE];

    while (fgets(line, sizeof(line), file) != NULL) {
        printf("%s", line);
    }

    printf("------------------\n");

    fclose(file);

    printf("Combined redirection verified successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano Skill15a.c
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ gcc Skill15a.c -o s15a
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ ./s15a
Combined Redirection System
---------------------------
Enter combined redirection: > combined.txt 2>&1
Output file: combined.txt
File opened successfully.

Standard streams restored successfully.

Integrated Output:
-----------------
Message 1: Standard output.
Message 2: Standard error.
Message 3: Standard output.
Message 4: Standard error.
Message 5: Standard output.
Message 6: Standard error.
-----------------
Combined redirection verified successfully.
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$
```

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_PARTS 20

/* Store one command in the execution plan */
typedef struct {
    char command[200];
    char input_file[100];
    char output_file[100];
    int has_input;
    int has_output;
} Command;

/* Remove leading spaces */
char *trim(char *str) {

    while (*str == ' ' || *str == '\t')
        str++;

    return str;
}

/* Parse one command and detect redirection */
void parseCommand(char *text, Command *cmd) {

    cmd->has_input = 0;
    cmd->has_output = 0;

    cmd->input_file[0] = '\0';
    cmd->output_file[0] = '\0';

    char temp[200];

    strcpy(temp, text);

    /* Check input redirection */
    char *input = strstr(temp, "<");

    if (input != NULL) {

        *input = '\0';

        input++;

        input = trim(input);

        sscanf(input, "%99s", cmd->input_file);

        cmd->has_input = 1;
```

```
GNU nano 8.7.1
        cmd->has_input = 1;
    }

    /* Check output redirection */
    char *output = strstr(temp, ">");

    if (output != NULL) {

        *output = '\0';

        output++;

        /* Handle >> */
        if (*output == '>')
            output++;

        output = trim(output);

        sscanf(output, "%99s", cmd->output_file);

        cmd->has_output = 1;
    }

    strcpy(cmd->command, trim(temp));
}

/* Validate command */
int validateCommand(Command *cmd) {

    if (strlen(cmd->command) == 0) {
        return 0;
    }

    if (cmd->has_input &&
        strlen(cmd->input_file) == 0) {
        return 0;
    }

    if (cmd->has_output &&
        strlen(cmd->output_file) == 0) {
        return 0;
    }

    return 1;
}

/* Display execution plan */
void displayPlan(Command commands[], int count) {

    printf("\nExecution Plan\n");
    printf("==============\n");
|
```

```
GNU nano 8.7.1
        printf("\nStep %d:\n", i + 1);

        printf("  Command : %s\n",
               commands[i].command);

        if (commands[i].has_input) {

            printf("  Input   : %s\n",
                   commands[i].input_file);
        }

        if (commands[i].has_output) {

            printf("  Output  : %s\n",
                   commands[i].output_file);
        }

        if (i < count - 1) {

            printf("  Pipe    : Connect output to next command\n");
        }
    }
}

int main() {

    char input[MAX_INPUT];

    Command commands[MAX_PARTS];

    printf("Complex Command Parser\n");
    printf("======================\n");

    printf("\nSupported examples:\n");
    printf("  cat input.txt | grep hello > output.txt\n");
    printf("  cat < input.txt | sort | wc -l > result.txt\n\n");

    printf("Enter command: ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Basic syntax validation */
    if (strlen(input) == 0) {

        printf("Error: Empty command.\n");
        return 1;
    }

    /* Check invalid pipe positions */
|   if (input[0] == '|' ||
```

```
GNU nano 8.7.1
    }

    /* Check invalid pipe positions */
    if (input[0] == '|' ||
        input[strlen(input) - 1] == '|') {

        printf("Error: Invalid pipe placement.\n");
        return 1;
    }

    /* Split command using pipe */
    int count = 0;

    char *part = strtok(input, "|");

    while (part != NULL &&
           count < MAX_PARTS) {

        parseCommand(part, &commands[count]);

        if (!validateCommand(&commands[count])) {

            printf("Error: Invalid command syntax.\n");
            return 1;
        }

        count++;

        part = strtok(NULL, "|");
    }

    printf("\nSyntax validation successful.\n");

    printf("Number of commands: %d\n", count);

    /* Generate execution plan */
    displayPlan(commands, count);

    printf("\nExecution order:\n");

    for (int i = 0; i < count; i++) {

        printf("%d -> %s\n",
               i + 1,
               commands[i].command);
    }

    printf("\nExecution plan generated successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:

sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano Skill15bc.c
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ nano Skill15b.c
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ gcc Skill15b.c -o s15b
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$ ./s15b
Complex Command Parser
======================

Supported examples:
  cat input.txt | grep hello > output.txt
  cat < input.txt | sort | wc -l > result.txt

Enter command: cat input.txt | grep hello > output.txt

Syntax validation successful.
Number of commands: 2

Execution Plan
==============

Step 1:
  Command : cat input.txt
  Pipe    : Connect output to next command

Step 2:
  Command : grep hello
  Output  : output.txt

Execution order:
1 -> cat input.txt
2 -> grep hello

Execution plan generated successfully.
sanjana3027@DESKTOP-QJF5AGU:      /SKILL$
```


sanjana3027@DESKTOP-QJF5AGU: /SKILL$ ./s15b
Complex Command Parser
Supported examples:
cat input.txt | grep hello > output.txt
cat < input.txt | sort | wc -l > result.txt
Enter command: cat < input.txt | sort | wc -l > result.txt
Syntax validation successful.
Number of commands: 3
Execution Plan
Step 1:
Command : cat
Input : input.txt
Pipe : Connect output to next command
Step 2:
Command : sort
Pipe : Connect output to next command
Step 3:
Command : wc -l
Output : result.txt
Execution order:
1 -> cat
2 -> sort
3 -> wc -l
Execution plan generated successfully.
sanjana3027@DESKTOP-QJF5AGU: /SKILL$